

DBMS – WEEK 5

Payment Transaction Management System

1. Scenario

The e-commerce company needs to maintain payment information for customer orders. The system should record payment details, track transaction status, manage successful and unsuccessful payments, and generate payment-related business reports. A Payment Transaction Module was created in Oracle SQL and connected to the existing Orders module through a foreign key.

2. Create Payment Table

The Payment entity stores customer payment transaction details. Each payment is linked to an order and to the product that was purchased.

Attribute	Description
Payment_ID	Unique payment identifier (primary key)
Order_ID	Order reference (foreign key to Orders)
Product_ID	Identifier of the product purchased
Product_Name	Name of the product purchased
Payment_Date	Date of transaction
Payment_Mode	UPI / Credit Card / Debit Card / Net Banking / Cash
Payment_Status	Completed / Pending / Failed
Transaction_Amount	Amount paid, NUMBER(10,2)

The table was created first, and the primary key and foreign key constraints were then added with ALTER TABLE.

```
SQL> CREATE TABLE Payment(Payment_ID INT, Order_ID VARCHAR2(8), Product_ID VARCHAR2(10), Product_Name VARCHAR2(15), Payment_Date DATE, Payment_Mode VARCHAR2(20), Payment_Status VARCHAR2(25), Transaction_Amount NUMBER(10,2));
Table created.
```

```
SQL> ALTER TABLE Payment
2 ADD CONSTRAINT PK_PAYMENT PRIMARY KEY(PAYMENT_ID);
Table altered.
```

3. Store Payment Details

Three payment transactions were inserted: a pending UPI payment and two completed card payments.

```
SQL> INSERT INTO PAYMENT
  2 (Payment_Id, Order_Id, Product_Id, Product_Name, Payment_Date,
  3 Payment_Mode, Payment_Status, Transaction_Amount)
  4 VALUES
  5 ('001', '0201', '101', 'RESIN KEYCHAIN',
  6 DATE'2026-08-22', 'UPI', 'Pending', 750);

1 row created.
```

```
SQL> INSERT INTO PAYMENT
  2 (Payment_Id, Order_Id, Product_Id, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3 VALUES
  4 ('002', '0202', '102', 'RESIN BOOKMARK', DATE '2026-08-23', 'CREDIT CARD', 'Completed', 1000);

1 row created.
```

```
SQL> INSERT INTO PAYMENT
  2 (Payment_Id, Order_Id, Product_Id, Product_Name, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  3 VALUES
  4 ('003', '0203', '103', 'Resin Pendant', DATE '2026-08-24', 'DEBIT CARD', 'Completed', 500);

1 row created.
```

4. Payment Records

The query `SELECT * FROM Payment` returns every stored payment record.

```
SQL> SELECT *FROM Payment;
```

PAYMENT_ID	ORDER_ID	PRODUCT_ID	PRODUCT_NAME	PAYMENT_D	PAYMENT_MODE
PAYMENT_STATUS	TRANSACTION_AMOUNT				
Pending	1 0201	101	RESIN KEYCHAIN	22-AUG-26	UPI
			750		
Completed	2 0202	102	RESIN BOOKMARK	23-AUG-26	CREDIT CARD
			1000		
Completed	3 0203	103	Resin Pendant	24-AUG-26	DEBIT CARD
			500		

5. Manage Successful and Pending Transactions

Payments were filtered by status to separate successful transactions from those still awaiting completion, and the payments in each status were counted.

Successful payments

```
SQL> SELECT *
2 FROM PAYMENT
3 WHERE Payment_Status = 'Completed';
```

PAYMENT_ID	ORDER_ID	PRODUCT_ID	PRODUCT_NAME	PAYMENT_D	PAYMENT_MODE
2	0202	102	RESIN BOOKMARK	23-AUG-26	CREDIT CARD
Completed			1000		
3	0203	103	Resin Pendant	24-AUG-26	DEBIT CARD
Completed			500		

Two payments are completed: Payment 2 (Credit Card, 1000) and Payment 3 (Debit Card, 500).

Pending payments

```
SQL> SELECT *
2 FROM PAYMENT
3 WHERE Payment_Status = 'Pending';
```

PAYMENT_ID	ORDER_ID	PRODUCT_ID	PRODUCT_NAME	PAYMENT_D	PAYMENT_MODE
1	0201	101	RESIN KEYCHAIN	22-AUG-26	UPI
Pending			750		

One payment is pending: Payment 1 (UPI, 750). No payment in the current data has the status Failed, so the unsuccessful-transaction check was run on the Pending status.

Count by status

```
SQL> SELECT Payment_Status, COUNT(Payment_Status)
2 FROM Payment
3 GROUP BY Payment_Status;
```

PAYMENT_STATUS	COUNT(PAYMENT_STATUS)
Pending	1
Completed	2

6. Analyze Payment Methods Used by Customers

Report 1: Payment Mode Analysis

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS
2  FROM PAYMENT
3  WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSACTIONS
-----
1
```

UPI transactions = 1.

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS
2  FROM PAYMENT
3  WHERE PAYMENT_MODE IN ('CREDIT CARD', 'DEBIT CARD');

CARD_PAYMENTS
-----
2
```

Card payments (credit and debit) = 2. Card payments are therefore the most common category, covering two of the three transactions.

Report 2: Revenue Analysis

Revenue is calculated from completed payments only.

```
SQL> SELECT
2  PAYMENT_MODE,
3  SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
4  FROM PAYMENT
5  WHERE PAYMENT_STATUS = 'Completed'
6  GROUP BY PAYMENT_MODE
7  ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE      TOTAL_REVENUE
-----
CREDIT CARD              1000
DEBIT CARD               500
```

Credit Card = 1000; Debit Card = 500.

```
SQL> SELECT
  2  ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVERAGE_TRANSACTION_AMOUNT
  3  FROM PAYMENT
  4  WHERE PAYMENT_STATUS = 'Completed';

AVERAGE_TRANSACTION_AMOUNT
-----
                          750
```

Average transaction amount = 750.

Report 3: Order Payment History

Orders were joined with Payment to show each order's dates, payment mode, amount and status, sorted by payment date.

```
SQL> SELECT
  2  O.ORDER_ID AS ORDER_ID,
  3  O.ORDER_DATE AS ORDER_DATE,
  4  P.PAYMENT_DATE AS PAYMENT_DATE,
  5  P.PAYMENT_MODE AS PAYMENT_MODE,
  6  P.TRANSACTION_AMOUNT AS AMOUNT,
  7  P.PAYMENT_STATUS AS PAYMENT_STATUS
  8  FROM ORDERS O
  9  JOIN PAYMENT P
 10  ON O.ORDER_ID = P.ORDER_ID
 11  ORDER BY P.PAYMENT_DATE;
```

ORDER_ID	ORDER_DAT	PAYMENT_D	PAYMENT_MODE	AMOUNT
0201	22-AUG-26	22-AUG-26	UPI	750
Pending				
0202	23-AUG-26	23-AUG-26	CREDIT CARD	1000
Completed				
0203	24-AUG-26	24-AUG-26	DEBIT CARD	500
Completed				

7. Payment Analysis

A combined report groups completed transactions by payment mode and gives the transaction count, total revenue and average transaction amount.

```

SQL> SELECT
  2     PAYMENT_MODE,
  3     COUNT(*) AS TRANSACTION_COUNT,
  4     SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE,
  5     ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION
  6 FROM PAYMENT
  7 WHERE PAYMENT_STATUS = 'Completed'
  8 GROUP BY PAYMENT_MODE
  9 ORDER BY TOTAL_REVENUE DESC;

```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
CREDIT CARD	1	1000	1000
DEBIT CARD	1	500	500

8. Conclusion

The Week 5 Payment Transaction Management System shows how Oracle SQL can be used to create a payment table linked to orders, store payment transactions, separate completed and pending payments, analyze the payment methods used, calculate revenue and generate an order payment history.

- Of three payments, two are completed (1500 in total) and one UPI payment of 750 is still pending.
- Credit Card produced the highest completed revenue (1000), followed by Debit Card (500).
- The average completed transaction amount is 750.